

Security Assessment Report

DWAO Accessibility Widget (dwao-a11y-widget.js)

OVERALL RESULT: PASSED — NO OPEN VULNERABILITIES

Component:	dwao-a11y-widget.js — client-side accessibility widget
Version assessed:	2.0.0
Prepared by:	DWAO
Assessment type:	Manual secure code review, automated static analysis (SAST) and automated runtime security testing
Classification:	Confidential

This report sets out the security parameters against which the DWAO Accessibility Widget was assessed, how each was verified, and the result. All 14 security parameters and all 17 runtime security tests passed.

1. Assessment Summary

14/14 Security parameters passed	17/17 Runtime security tests passed	0 SAST errors	0 Open vulnerabilities
--	---	-------------------------------------	--

The DWAO Accessibility Widget is a single, self-contained JavaScript file that adds an accessibility toolbar (contrast, text sizing, screen reader, page structure, magnifier, virtual keyboard and more) to a website. It is added to a page with one `<script>` tag and has no third-party dependencies.

The widget was assessed against 14 security parameters aligned to the OWASP Top 10 (2021) and secure client-side coding practice. **It passed every parameter.** Key results:

- **No data leaves the browser.** The widget makes no network requests of any kind.
- **No personal data is collected or stored.** The only thing saved is the visitor's own choice of accessibility options.
- **Passwords are never exposed.** No widget feature displays, magnifies, speaks or stores password values.
- **Protected against cross-site scripting.** Website content is always displayed as plain text, verified with real attack payloads.
- **No supply-chain exposure.** Zero third-party libraries or external scripts.

Severity	Open issues
Critical	0
High	0
Medium	0
Low	0

2. Scope & Assessment Approach

2.1 Scope

The distributable file `dwao-a11y-widget.js` (v2.0.0) and its integration artefacts (`test-page.html`, `INTEGRATION-GUIDE.html`). The security of the websites that embed the widget is outside this scope.

2.2 Approach

The widget was verified in three complementary ways:

Method	What it covers
Manual secure code review	The full source was read line by line. Every DOM write, storage access, form-field access and configuration input was traced to its data source.
Automated static analysis (SAST)	ESLint 9 with Mozilla's <code>eslint-plugin-no-unsanitized</code> (HTML-injection sinks), <code>eslint-plugin-security</code> (recommended rule set) and core rules <code>no-eval</code> , <code>no-implied-eval</code> , <code>no-new-func</code> , <code>no-script-url</code> , <code>no-debugger</code> and <code>no-console</code> .
Automated runtime security testing	The widget was loaded into a deliberately hostile test page in a simulated browser (jsdom) containing XSS payloads in image alt text and headings, a filled password field, a malicious configuration value and tampered saved settings. Each feature was switched on and its behaviour checked, while all network channels were monitored. The test suite was also run against a deliberately weakened copy of the widget to confirm it detects real vulnerabilities.

2.3 Standards reference

Parameters are mapped to the OWASP Top 10 (2021) categories where applicable, and to general secure client-side JavaScript practice otherwise.

3. Security Parameters Checklist

Each parameter below was verified by code review and, where marked with a test ID, by automated runtime tests (section 4).

#	Security parameter	Reference	What was verified	Result
1	Cross-site scripting (DOM XSS)	A03 Injection	Every place the widget writes to the page was traced to its data source. All page-derived text (image alt text, headings, form placeholders, voice names) is written with <code>textContent</code> or text nodes and cannot be interpreted as HTML. Confirmed with hostile payloads at runtime (T-01, T-02, T-14).	PASS
2	HTML / CSS injection via configuration	A03 Injection	Script-tag settings are either matched against a fixed list (data-position, data-theme) or strictly validated (data-brand-color accepts hex colours only), so they cannot inject CSS or markup. A malicious brand-colour value is rejected (T-05).	PASS
3	Dynamic code execution	A03 Injection	No <code>eval()</code> , <code>new Function()</code> , string-based timers, <code>document.write</code> or <code>javascript:</code> URLs. Enforced by automated static rules and a runtime tripwire (T-13).	PASS
4	Network communication & data exfiltration	Privacy / Data exfiltration	The widget makes no network requests of any kind: no <code>fetch</code> , <code>XMLHttpRequest</code> , <code>sendBeacon</code> , <code>WebSocket</code> , <code>postMessage</code> or external resource loads. Verified by monitoring all channels during a full session (T-12).	PASS
5	Client-side data storage	A04 Insecure Design	Only one <code>localStorage</code> key is used, holding a list of the visitor's selected accessibility options. No <code>sessionStorage</code> , no cookies, no <code>IndexedDB</code> (T-07, T-08).	PASS
6	Personal & sensitive data (PII)	A02 Cryptographic Failures / Privacy	The widget does not collect, store or transmit personal data. Nothing typed into the host page's forms is stored or transmitted, and password values never appear in any widget UI (T-07, T-11).	PASS
7	Password field handling	A07 Authentication Failures	Password values are never read for display, magnified, spoken or stored. The only change made to password fields is restoring paste support for password managers, which leaves the value untouched (T-03, T-10).	PASS
8	Hard-coded secrets & credentials	A02 Cryptographic Failures	No API keys, tokens, passwords, internal URLs or credentials anywhere in the source.	PASS
9	Third-party dependencies & supply chain	A06 Vulnerable Components	Zero dependencies: no npm packages, no bundled libraries, no CDN includes. The shipped file is the entire attack surface.	PASS
10	Tampered stored data	A08 Data Integrity	Saved settings are type-checked and escaped before use, so a tampered <code>localStorage</code> value cannot affect the page. Verified with a crafted payload (T-06).	PASS
11	Global namespace & isolation	Secure coding	All code runs in a single strict-mode closure. Exactly one global is exposed: <code>window.DWA0Accessibility</code> (version and <code>reset()</code>). Verified at runtime (T-15).	PASS
12	Host-page integrity	Secure coding	Changes to the host page are limited to accessibility improvements (ARIA labels, <code>iframe</code> titles, <code>lang</code> attribute, skip link, visual classes) and are fully reversible via <code>Reset</code> (T-09, T-16). Form submission, links and page logic are not altered.	PASS

#	Security parameter	Reference	What was verified	Result
13	Error handling & resilience	Secure coding	Feature code is guarded so that unusual or malformed host-page content cannot break the host page or the widget. The widget loads cleanly on a deliberately hostile page (T-00).	PASS
14	Debug artefacts & code hygiene	Secure coding	No debugger statements, console logging, test hooks or commented-out credentials. Enforced by automated rules (no-debugger, no-console).	PASS

4. Runtime Security Test Results

Automated tests run against the final v2.0.0 source in a simulated browser, on a test page seeded with attack payloads. 17 of 17 passed.

Test	Check	Result
T-00	Widget loads without errors on a hostile page	PASS
T-01	Image alt text rendered as plain text (no HTML injection)	PASS
T-02	Page headings rendered as plain text in Page Structure (no HTML injection)	PASS
T-03	Magnifier never displays password field contents	PASS
T-04	Magnifier still magnifies normal text fields (no functional regression)	PASS
T-05	Malicious data-brand-color value rejected (no CSS injection)	PASS
T-06	Poisoned localStorage value handled safely; valid saved setting restored	PASS
T-07	Only the visitor's selected option names are stored (no PII, no page content, no password)	PASS
T-08	No cookies read or written	PASS
T-09	Skip link inserted even when the page has a malformed in-page anchor	PASS
T-10	Password paste re-enabled without reading or altering the password	PASS
T-11	Password value never copied into any widget UI element	PASS
T-12	No network requests (fetch, XHR, beacon, WebSocket, postMessage)	PASS
T-13	No dynamic code execution at runtime (eval)	PASS
T-14	No injected markup executed anywhere on the page	PASS
T-15	Public API exposed (window.DWAOAccessibility) with version and reset()	PASS
T-16	Reset restores page without errors	PASS

5. Static Analysis (SAST) Results

Rule set	Checks for	Errors	Result
no-unsanitized (Mozilla)	Unsafe innerHTML / outerHTML / insertAdjacentHTML / document.write	0	PASS
Core ESLint security rules	eval, implied eval, new Function, javascript: URLs	0	PASS

Rule set	Checks for	Errors	Result
eslint-plugin-security	Unsafe regex, child processes, non-literal requires, timing attacks, object injection	0	PASS
Code hygiene rules	debugger statements, console output	0	PASS

eslint-plugin-security also raised 12 advisory "object injection" notices, a rule that flags every obj[key] lookup. Each was reviewed manually: every one uses a fixed widget label or a numeric index the widget computes itself, with no path for external input. No action required.

6. Security Hardening Completed

As part of this assessment the following improvements were identified and implemented in v2.0.0, and each was confirmed by the runtime tests above. There are no outstanding items.

Area	Improvement implemented	Verified by	Status
Image alt-text tooltip	Alt text is displayed strictly as plain text, preventing HTML injection from website content.	T-01, T-14	Resolved
Page Structure panel	Heading text is displayed strictly as plain text, preventing HTML injection from website content.	T-02, T-14	Resolved
Text magnifier	Password fields are excluded, so password values are never displayed in the magnifier lens.	T-03, T-04	Resolved
Saved settings	Values read back from browser storage are type-checked and escaped before use.	T-06	Resolved
Brand-colour setting	Only valid hex colours are accepted from the script-tag configuration.	T-05	Resolved
Skip link	Skip-link insertion is resilient to malformed in-page anchors on the host site.	T-09	Resolved
Internal state	Internal variables moved into the widget's private scope; one public global only.	T-15	Resolved
Safe DOM construction	Icons and labels are built with safe DOM APIs; static analysis reports zero HTML-injection sinks.	SAST	Resolved
Code hygiene	Leftover debugger statement removed; file header and version reference aligned to v2.0.0.	SAST	Resolved

7. Data Handling & Privacy Statement

Question	Answer
Does the widget send any data to a server?	No. It makes no network requests.
Does it collect personal data?	No.
Does it use cookies?	No.
What does it store?	One browser localStorage entry listing the visitor's selected accessibility options (e.g. "highContrast"). It is cleared by Reset.
Does it read form input?	Only to magnify a non-password field's text on hover, or to type into it with the on-screen keyboard. Values are never stored or sent.
Does it access password fields?	Only to restore paste support, and to type the characters a visitor clicks on the on-screen keyboard. Password values are never displayed, stored or sent.
Does it load external code?	No. Zero third-party libraries or external scripts.

The Read Page feature uses the browser's built-in speechSynthesis API. Speech is produced by the visitor's own browser or operating system using the voice they select; the widget itself sends nothing.

8. Conclusion

RESULT: PASSED

The DWAOW Accessibility Widget v2.0.0 was assessed against 14 security parameters covering injection, data exfiltration, data storage, sensitive-data handling, credentials, supply chain, isolation and code hygiene. **It passed every parameter**, all 17 automated runtime security tests, and automated static analysis with zero errors.

The widget has a deliberately small attack surface: it is a single dependency-free file, it never communicates with a server, it stores no personal data, and it treats all website content as plain text. No open vulnerabilities of any severity were identified at the time of assessment.

The widget is assessed as suitable for deployment on production websites, including those that handle financial or personal information.

Maintaining this result

- Re-run the static analysis and runtime security tests on every future change to the widget.
- Deliver the widget from the same origin as the host site, or with a Subresource Integrity (SRI) hash, so the file cannot be altered in transit or on a third-party host.